Semantic structure

13 min read Updated 15 May 2026 Topic 1 of 77 Bookmark Mark done

1A. Semantic Structure

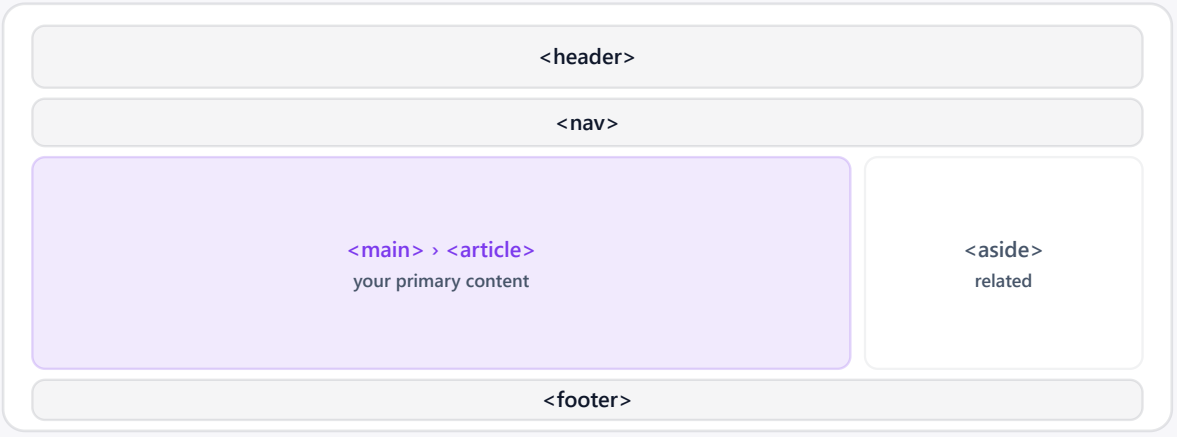
Questions: 4

Difficulty: Junior to Mid

Companies: Google, Amazon, Flipkart, Microsoft, Swiggy, Atlassian, Adobe, Razorpay

Q 1.1 Why does semantic HTML matter? Can't we just use divs for everything?

A semantic page skeleton



Landmark elements give the same layout meaning — screen readers, SEO crawlers, and browsers all understand it. A wall of s conveys none of this.

Difficulty: Junior

Asked at: Flipkart, Amazon, Razorpay, Google

You technically can build an entire page using `<div>` and `<span>` tags. It'll look exactly the same visually. But the moment you step outside the visual browser, everything breaks.

Think about it — a screen reader user lands on your page. If everything is a `<div>`, the screen reader has no idea what's a navigation menu, what's the main content, and what's a sidebar. It reads your page like a long, unstructured wall of text. There's no way to jump between sections, skip to the main content, or understand the page layout. The user is stuck listening to every single element in order.

Semantic tags solve this by telling the browser (and assistive technologies) what each piece of content actually *is*.

HTML Copy

```html
<!-- This tells nothing about the page structure -->
<div class="header">
  <div class="nav">
    <div class="nav-item">Home</div>
    <div class="nav-item">Products</div>
  </div>
</div>
<div class="content">
  <div class="article">
    <div class="title">Understanding Closures</div>
    <div class="body">Closures are one of the most ... </div>
  </div>
</div>
<div class="footer">
  <div class="links">Contact us</div>
</div>

<!-- This tells everything -->
<header>
  <nav aria-label="Main navigation">
    <a href="/">Home</a>
    <a href="/products">Products</a>
  </nav>
</header>
<main>
  <article>
    <h1>Understanding Closures</h1>
```

```
</main>
<footer>
  <a href="/contact">Contact us</a>
</footer>
```

A screen reader user on the second version can press a shortcut key to jump directly to `<nav>`, skip to `<main>`, or list all `<article>` elements on the page. They navigate the page as efficiently as a sighted user scanning it visually.

Beyond accessibility, semantic HTML impacts SEO. Search engines use heading hierarchy (`h1` through `h6`), `<article>`, `<section>`, and `<nav>` to understand page structure and content relevance. Two pages with identical text content will rank differently if one uses proper semantic markup and the other uses a flat soup of divs.

There's also a maintainability argument. When a developer opens a file six months later and sees `<nav>`, they instantly know what it does. When they see `<div class="sc-fKgJPI hYzrOF">`, they have to dig through CSS class mappings or inspect the DOM to figure it out.

Follow-up interviewers ask: "Name five semantic tags you use regularly and explain when you'd pick one over the other."

Good answer: `<header>`, `<nav>`, `<main>`, `<section>`, `<article>`, `<aside>`, `<footer>`. The key distinction most people miss — `<article>` is for self-contained content that makes sense on its own (a blog post, a product card, a comment), while `<section>` is for thematically grouped content that needs a heading but isn't standalone. If you can take the content out and put it on another page and it still makes sense, it's an `<article>`. If it only makes sense in the context of the surrounding page, it's a `<section>`.

Common mistake: Using `<section>` as a generic wrapper the same way you'd use `<div>`. A `<section>` without a heading is semantically meaningless — just use a `<div>` for layout wrappers.

Q 1.2 What's the difference between `<section>`, `<article>`, and `<div>`? When do you use which?



Reach for a meaningful element first; fall back to only for layout wrappers.

Difficulty: Junior

Asked at: Swiggy, Atlassian, Walmart, Adobe, PhonePe

This trips up a lot of candidates because the differences feel subtle until you have the right mental model. Here's how to think about it:

`<article>` — Ask yourself: can this piece of content be taken out of the page and placed somewhere else (a different page, an RSS feed, a social media embed) and still make complete sense? If yes, it's an article. A blog post is an article. A product card is an article. A user comment is an article. A tweet is an article. Each one is independently meaningful.

`<section>` — Ask yourself: is this a thematic grouping of content that needs a heading? If yes, use section. The "Features" section of a landing page. The "Reviews" section of a product page. The "Education" section of a resume page. A section always has a heading. If you can't meaningfully put a heading on it, it probably shouldn't be a `<section>`.

`<div>` — Is there no semantic meaning at all, and you just need a container for styling or layout purposes? That's a div. A wrapper for flexbox layout. A container for CSS grid. A grouping element for applying a background color. These are divs.

HTML Copy

```
<main>
  <!— Section: thematic grouping with a heading —>
```

```
    <!-- Article: self-contained, makes sense alone -->
    <article>
      <h3>Understanding Closures</h3>
      <p>Closures are one of the most misunderstood concepts ... </p>
      <time datetime="2026-03-15">March 15, 2026</time>
    </article>

    <article>
      <h3>CSS Grid Deep Dive</h3>
      <p>Grid layout changed how we think about ... </p>
      <time datetime="2026-03-12">March 12, 2026</time>
    </article>
  </section>

  <aside>
    <section aria-labelledby="tags-heading">
      <h2 id="tags-heading">Popular Tags</h2>
      <!-- Div: no semantic meaning, just a layout wrapper -->
      <div class="tag-cloud">
        <a href="/tag/react">React</a>
        <a href="/tag/css">CSS</a>
        <a href="/tag/typescript">TypeScript</a>
      </div>
    </section>
  </aside>
</main>
```

Notice how articles live inside a section, and the div is purely structural. This hierarchy tells screen readers and search engines exactly how the content is organized — there's a section of latest articles, each article is self-contained, and there's a sidebar with popular tags.

Can articles contain sections? Yes. A long blog post (an `<article>`) might have multiple sections within it — "Introduction", "The Problem", "The Solution", "Conclusion". Each section groups related content within the article.

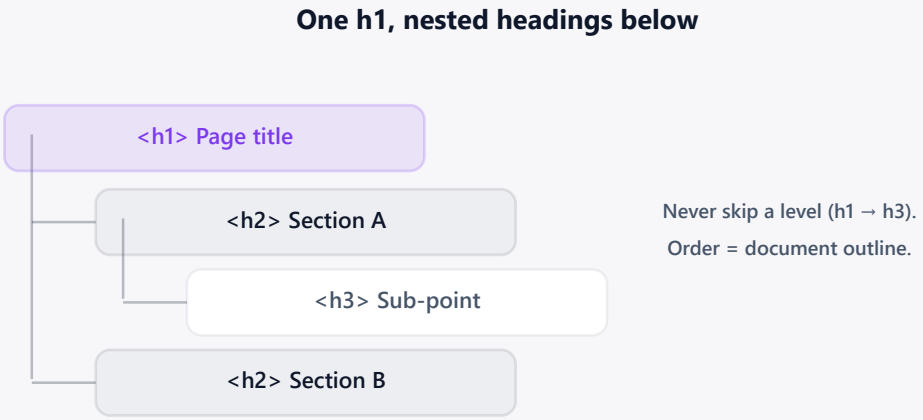
Can sections contain articles? Yes. That's the pattern shown above — a "Latest Articles" section contains multiple articles.

When people get confused: They treat `<section>` as a replacement for `<div>`. If you're wrapping a group of elements just for styling — flexbox container, grid wrapper, background color group — use `<div>`. Semantic tags are for meaning, not layout.

Follow-up: "What about `<aside>`? When do you use that?"

`<aside>` is for content that's tangentially related to the main content — a sidebar, a pull quote, an ad, a related links widget. It's content that could be removed and the main content would still make complete sense. Screen readers let users skip `<aside>` content entirely.

Q 1.3 How do you build a proper heading hierarchy and why does it matter?



Exactly one h1 per page, then descend one level at a time. This outline is how assistive tech and search engines navigate your content.

Difficulty: Junior-Mid
Asked at: Microsoft, Amazon, Adobe, Atlassian

Screen reader users have a shortcut to list all headings on a page and jump between them. In JAWS, it's the H key. In NVDA, the same. In VoiceOver, you open the Rotor and select Headings. This is one of the most common navigation methods for screen reader users — roughly 70% use headings as their primary way to find content on a page.

If your heading hierarchy is broken — skipping levels, using headings for styling, or not using headings at all — this navigation method becomes useless.

HTML Copy

```html
<!— Correct: No skipped levels, logical nesting —>
<h1>Product Dashboard</h1>

  <h2>Sales Overview</h2>
    <h3>Monthly Revenue</h3>
    <h3>Top Products</h3>

  <h2>Customer Analytics</h2>
    <h3>New Customers</h3>
    <h3>Retention Rate</h3>
      <h4>By Region</h4>
      <h4>By Product Category</h4>


<!— Wrong: Skipped levels, heading used for styling —>
<h1>Dashboard</h1>
<h4>Sales</h4>            <!— Skipped h2, h3 - screen reader users lose co
<h2>Big Bold Text</h2>    <!— Used for visual size, not document structure
<h2>Another Section</h2>
```

The rules:

One `<h1>` **per page.** This is the page title — the most important heading. In a single-page app, update the `<h1>` when the route changes. On a product listing page, the `<h1>` might be "Running Shoes". On the product detail page, it might be "Nike Air Max 270".

Don't skip levels. Go h1 → h2 → h3, not h1 → h3. Skipping levels confuses screen readers and implies a structural relationship that doesn't exist. You can go back up levels freely — after an h3, the next heading can be an h2 (starting a new section).

Don't use headings for visual styling. If you need text to look bigger or bolder, use CSS. The heading level should reflect the document structure, not the visual design. A common mistake is using `<h3>` for a card title because it's the right font size. Instead:

HTML Copy

```
  <h3>Featured Product</h3>
</div>

<!-- Good: heading level matches document structure, CSS handles visual si
<div class="card">
  <h2 class="card-title">Featured Product</h2>
</div>
```

Components should accept heading levels as props. When you build a reusable Card or Section component, the correct heading level depends on where the component is used in the page. A Card at the top level might need `<h2>`, but the same Card inside a sidebar section might need `<h3>`:

JSX Copy

```jsx
// Bad: Hardcoded heading level — breaks hierarchy when used in different
function Card({ title, children }) {
  return (
    <div className="card">
      <h2>{title}</h2>
      {children}
    </div>
  );
}

// Good: Heading level is configurable
function Card({ title, headingLevel = 2, children }) {
  const Heading = `h${headingLevel}`;
  return (
    <div className="card">
      <Heading>{title}</Heading>
      {children}
    </div>
  );
}

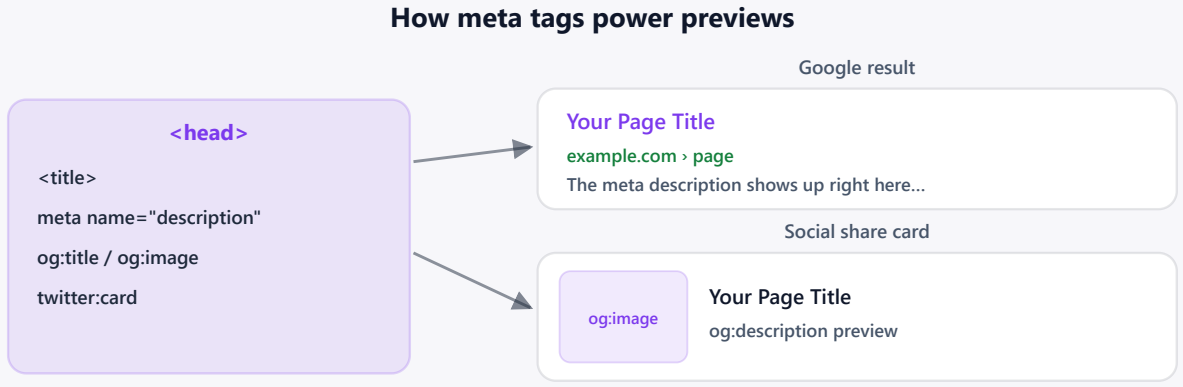
// Usage
<section>
```

```
</section>
```

Follow-up: "How do you check if your heading hierarchy is correct?"

The quickest way is to install the HeadingsMap browser extension — it shows the heading tree for any page. You can also use Lighthouse's accessibility audit, which flags skipped heading levels. In code, the `eslint-plugin-jsx-a11y` rule `heading-has-content` and manual review catch most issues.

Q 1.4 What are meta tags, Open Graph tags, and how does HTML affect SEO?



The same tags in <head> feed both search snippets and social share cards — get them right and every link to your site looks intentional.

Difficulty: Mid
Asked at: Flipkart, Swiggy, Amazon, Razorpay

Meta tags provide metadata about the page to browsers, search engines, and social media platforms. They don't affect what users see on the page, but they significantly affect how the page appears in search results, social shares, and browser behavior.

Essential meta tags:

```
<head>
  <!-- Character encoding - always first, always UTF-8 -->
  <meta charset="UTF-8" />

  <!-- Viewport - required for responsive design -->
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />

  <!-- Page title - shows in browser tab, search results, social shares -
  <title>Nike Air Max 270 - Running Shoes | ShoeStore</title>

  <!-- Description - shows in search result snippets (150-160 chars) -->
  <meta name="description" content="Shop the Nike Air Max 270. Lightweight

  <!-- Robots - tells search engines how to index -->
  <meta name="robots" content="index, follow" />

  <!-- Canonical URL - prevents duplicate content issues -->
  <link rel="canonical" href="https://shoestore.com/nike-air-max-270" />

  <!-- Language -->
  <html lang="en">
</head>
```

Open Graph tags (Facebook, LinkedIn, WhatsApp):

When someone shares your URL on social media, these tags control what the preview card looks like:

HTML Copy

```
<meta property="og:title" content="Nike Air Max 270 - Running Shoes" />
<meta property="og:description" content="Lightweight running shoe with Max
<meta property="og:image" content="https://shoestore.com/images/air-max-27
<meta property="og:url" content="https://shoestore.com/nike-air-max-270" /
<meta property="og:type" content="product" />
<meta property="og:site_name" content="ShoeStore" />
```

Twitter Card tags:

```
<meta name="twitter:card" content="summary_large_image" />
<meta name="twitter:title" content="Nike Air Max 270 – Running Shoes" />
<meta name="twitter:description" content="Lightweight running shoe with Ma
<meta name="twitter:image" content="https://shoestore.com/images/air-max-2
```

How HTML structure affects SEO:

Search engines use your HTML structure to understand what your page is about and how important each piece of content is:

Heading hierarchy tells importance. Your `<h1>` is the most important text on the page. `<h2>` headings are section-level important. Google uses this hierarchy to understand your content structure.

Semantic tags help categorization. `<article>` tells Google "this is a self-contained piece of content." `<nav>` tells it "these are navigation links, not content." `<aside>` tells it "this is supplementary, not primary content."

`<a>` **tag text matters.** "Click here" as link text tells Google nothing. "View Nike Air Max 270 reviews" tells Google exactly what the linked page contains.

HTML Copy

```
<!-- Bad for SEO -->
<a href="/reviews/air-max-270">Click here</a>

<!-- Good for SEO -->
<a href="/reviews/air-max-270">Read 234 reviews for Nike Air Max 270</a>
```

Structured data (JSON-LD) gives rich results:

HTML Copy

```
<script type="application/ld+json">
{
  "@context": "https://schema.org",
  "@type": "Product",
```

```
  "brand": { "@type": "Brand", "name": "Nike" },
  "offers": {
    "@type": "Offer",
    "price": "12995",
    "priceCurrency": "INR",
    "availability": "https://schema.org/InStock"
  },
  "aggregateRating": {
    "@type": "AggregateRating",
    "ratingValue": "4.5",
    "reviewCount": "234"
  }
}
</script>
```

This structured data tells Google exactly what the page contains, enabling rich search results with star ratings, prices, and availability shown directly in search results.

For SPAs (React/Next.js):

Client-side rendered pages have an SEO problem — Google's crawler might see an empty `<div id="root">` because JavaScript hasn't executed yet. Solutions:

- **Server-side rendering (Next.js)** — HTML is rendered on the server, so crawlers see full content
- **Static generation** — Pages are pre-built at build time with all content
- **Dynamic meta tags** — Use `react-helmet` or Next.js `<Head>` to set per-page meta tags:

JSX Copy

```jsx
// Next.js App Router
export const metadata = {
  title: "Nike Air Max 270 - Running Shoes | ShoeStore",
  description: "Lightweight running shoe with Max Air unit for all-day com
  openGraph: {
    title: "Nike Air Max 270",
    images: ["/images/air-max-270-og.jpg"],
```

Follow-up: "What's the viewport meta tag and what happens without it?"

Without `<meta name="viewport" content="width=device-width, initial-scale=1.0">`, mobile browsers render the page as if it's a desktop page (typically 980px wide) and then shrink it down to fit the screen. The user sees a tiny zoomed-out version of the desktop site. The viewport meta tag tells the browser to match the viewport width to the device width and start at 1x zoom — which is the foundation of responsive design.

QUICK CHECK — no marks, just for you.

1. You are marking up a user comment on a blog post. According to the lesson's mental model, which element is the right choice for the comment itself?

- ○ aside, because a comment is tangential to the main post content
- ○ section, because the comment is a thematic grouping that belongs under the comments heading
- ○ article, because the comment is self-contained and still makes sense if lifted out of the page
- ○ div, because a comment carries no semantic meaning of its own

Check answer

2. A page has an h1, then an h2, then an h3. The author now wants to start a brand-new top-level section. Per the lesson's heading rules, what is the correct next heading and why?

- ○ An h2 — you may move back up levels freely; only skipping levels downward is the problem
- ○ A second h1 — each new top-level section on the page gets its own h1

An h4 — every subsequent heading must continue one level deeper than the previous one

- ◯ An h3 — once you reach h3 you must stay at h3 or deeper for the rest of the page

Check answer

3. The lesson argues a reusable Card component should take its heading level as a prop instead of hardcoding an h2. What is the reasoning?

- ◯ The correct heading level depends on where the component sits in the page, so a hardcoded level breaks the hierarchy in some contexts
- ◯ React cannot render an h2 inside a div, so the tag name must be computed at runtime
- ◯ Screen readers ignore any heading that a component renders unless its level was passed explicitly as a prop
- ◯ Hardcoded heading tags cannot be styled with CSS, so the level must be passed in to control font size

Check answer

Next ›
Html5 apis and forms

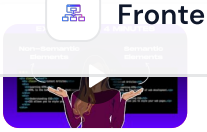
LEARN MORE

ARTICLES & DOCS

Semantics (MDN glossary)
MDN

VIDEOS

Frontend System Desi...



Semantic HTML in 4 minutes

YouTube



Semantic HTML elements tutorial

YouTube



Semantic tags explained simply

YouTube

Hand-picked references for this lesson.